Practical – 5

Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

1. What is Decision Tree?

Decision Trees (DTs) are a non-parametric supervised learning method used for classification and regression. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. A tree can be seen as a piecewise constant approximation.

It splits the dataset into smaller subsets based on feature values, forming a structure similar to a flowchart.

- Internal Nodes → Represent decision based on an attribute
- Branches → Represent outcome of the decision
- Leaf Nodes → Represent final class label Decision Trees are easy to

understand and interpret.

2. What is ID3 Algorithm?

ID3 (Iterative Dichotomiser 3) is a Decision Tree algorithm developed by Ross Quinlan.

It is mainly used for classification problems.

The ID3 algorithm builds the tree by selecting the best attribute using Information Gain.

3. Core Concept of ID3

ID3 uses two important concepts:

(i) Entropy

Entropy measures the impurity or randomness in the dataset.

Formula:

Entropy(S)=−∑pilog2piEntropy(S) = - \sum p_i \log_2 p_iEntropy(S)=−∑pi log2pi

Where:

- pip_ipi = Probability of class i
- If entropy = 0 → Data is pure
- If entropy = 1 → Data is highly impure

(ii) Information Gain

Information Gain tells us how much information a feature gives about the class.

Formula:

InformationGain=Entropy(S)−∑|Sv||S|Entropy(Sv)Information Gain = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)InformationGain=Entropy(S)−∑|S||Sv|Entropy(Sv) Where:

- S = Original dataset
- SvS_vSv = Subset after splitting on attribute

The attribute with the highest Information Gain is selected as the root node.

4. How ID3 Algorithm Works?

Core Idea:

- Start with the complete dataset
- Calculate entropy of dataset
- Calculate Information Gain for each attribute
- Select the attribute with highest Information Gain
- Split the dataset based on that attribute
- Repeat the process for each subset

- Stop when:
  - All samples belong to same class OR

    No attributes are left

The final result is a fully constructed decision tree.

5. Steps of ID3 Algorithm Implementation

Step 1: Dataset Preparation

Select an appropriate classification dataset containing multiple attributes and a target class label. import pandas as pd import numpy as np from math import log2

```
dataset = {
   'Outlook': ['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast',
               'Sunny','Sunny','Rain','Sunny','Overcast','Overcast','Rain'],
   'Temp': ['Hot','Hot','Hot','Mild','Cool','Cool','Cool',
            'Mild','Cool','Mild','Mild','Mild','Hot','Mild'],
   'Humidity': ['High','High','High','High','Normal','Normal','Normal',
                'High','Normal','Normal','Normal','High','Normal','High'],
   'Wind': ['Weak','Strong','Weak','Weak','Weak','Strong','Strong',
            'Weak','Weak','Weak','Strong','Strong','Weak','Strong'],
   'Play': ['No','No','Yes','Yes','Yes','No','Yes',
            'No','Yes','Yes','Yes','Yes','Yes','No']
}
```

```
df = pd.DataFrame(dataset)

print(df)
```

Output:

```
...       Outlook  Temp Humidity    Wind Play
    0       Sunny   Hot     High    Weak   No
    1       Sunny   Hot     High  Strong   No
    2    Overcast   Hot     High    Weak  Yes
    3        Rain  Mild     High    Weak  Yes
    4        Rain  Cool   Normal    Weak  Yes
    5        Rain  Cool   Normal  Strong   No
    6    Overcast  Cool   Normal  Strong  Yes
    7       Sunny  Mild     High    Weak   No
    8       Sunny  Cool   Normal    Weak  Yes
    9        Rain  Mild   Normal    Weak  Yes
    10      Sunny  Mild   Normal  Strong  Yes
    11   Overcast  Mild     High  Strong  Yes
    12   Overcast   Hot   Normal    Weak  Yes
    13       Rain  Mild     High  Strong   No
```

Step 2: Calculate Entropy

Compute entropy for the entire dataset to measure impurity.

```
def entropy(target_col):
  values, counts = np.unique(target_col,return_counts=True)
  entropy_value = 0
  for i in range(len(values)):
    p = counts[i] / sum(counts)
    entropy_value -=p * log2(p)
  return entropy_value
print("Entropy of dataset:", entropy(df['Play']))
```

Output:

```
•••  Entropy of dataset: 0.9402859586706311
```

Step 3: Calculate Information Gain

For each attribute:

- Split dataset based on attribute values
- Calculate entropy of each subset
- Compute Information Gain

Select the attribute with highest gain.

```
def information_gain(data, feature, target="Play"):
    total_entropy = entropy(data[target])
    values, counts = np.unique(data[feature], return_counts=True)
    weighted_entropy = 0
    for i in range(len(values)):
        subset = data[data[feature] == values[i]]
        weighted_entropy += (counts[i] / sum(counts)) * entropy(subset[target])
        return total_entropy - weighted_entropy
for col in df.columns[:-1]:
    print(col, ":", information_gain(df, col))
```

Output:

```
Outlook : 0.9402859586706311
Temp : 0.7084922088251646
Humidity : 0.4476718906535054
Wind : 0.5117145300992025
```

Step 4: Build the Tree

- Make selected attribute as root node
- Create branches for each possible value
- Recursively repeat the process for each branch

```
def id3(data, original_data, features, target="Play", parent_node=None):
    if len(np.unique(data[target])) <= 1:
        return np.unique(data[target])[0]
    elif len(data) == 0:
        return(np.unique(original_data[target])[np.argmax(np.uniq
        ue(original_data[target], return_counts=True)[1])])
    elif len(features) == 0:
        return parent_node
    else:
        parent_node=np.unique(data[target])[np.argmax(np.uniqu
        e(data[target],return_counts=True)[1])]
        gains = [information_gain(data, feature, target) for feature in features]
        best_feature = features[np.argmax(gains)]
        tree = {best_feature: {}}
        features = [f for f in features if f != best_feature]
        for value in np.unique(data[best_feature]):
            sub_data = data[data[best_feature] == value]
            subtree = id3(sub_data, original_data, features, target,
            parent_node)
```

```
        tree[best_feature][value] = subtree
        return tree
```

Step 5: Stopping Criteria

Stop building the tree when:

- All samples in subset belong to same class
- No attributes are available
- Dataset becomes empty

```
features = df.columns[:-1].tolist()
tree = id3(df, df, features)
print("Decision Tree:")
print(tree)
```

Output:

```
Decision Tree:
{'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}, 'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}}}
```

Step 6: Classification of New Sample To classify a new sample:

- Start from root node
- Follow the branch according to feature value
- Continue until reaching a leaf node
- The leaf node represents predicted class

```
def predict(query, tree):
    for key in list(query.keys()):
        if key in tree:
            value = query[key]
subtree = tree[key][value]
if isinstance(subtree, dict):
            return predict(query, subtree)
else:
        return subtree
new_sample = {
    'Outlook': 'Rain',
    'Temp': 'Mild',
    'Humidity': 'High',
    'Wind': 'Weak'
}
result = predict(new_sample, tree)
print("Prediction:", result)
```

Output:

```
Prediction: Yes
```

6. Advantages of ID3

- Simple and easy to understand
- Works well with categorical data
- No need for data normalization
- Fast computation

7. Limitations of ID3

- Can overfit the training data
- Cannot handle continuous data directly
- Biased toward attributes with many values
- No pruning mechanism

8. Applications of Decision Tree (ID3)

- Medical diagnosis
- Customer classification
- Spam detection
- Credit risk analysis
- Weather prediction

Conclusion

The ID3 algorithm is a popular Decision Tree method used for classification tasks. It constructs a tree structure using Entropy and Information Gain to select the best attributes. The generated tree can be used to classify new samples by traversing from root to leaf node.